EndtermRenwed\Group5\client.ino

```
/**
 * @file client.ino
 * @brief Client-side code for a two-factor authentication (2FA) secure vault system.
 * * This Arduino sketch controls the client-side hardware of a secure vault.
 * It communicates with a central server via an nRF24L01 radio module.
 * * --- Features ---
 * 1.  **Serial Input:** Accepts user commands (password, 'close', 'reset') via the Serial Monitor.
 * 2.  **RF Communication:** Sends requests to and receives commands/responses from a server.
 * 3.  **First Factor Auth:** Sends a user-entered password to the server for validation.
 * 4.  **Second Factor Auth:** Initiates a simple dot matrix game controlled by an IR remote.
 * 5.  **Servo Control:** Operates a local servo motor to open/close the vault door.
 * 6.  **Dot Matrix Display:** Provides visual feedback to the user (e.g., "PSWRD", "OK", "OPEN").
 * 7.  **Emergency Override:** Listens for a "LOCKDOWN" command from the server to force the vault closed.
 *
 * --- Hardware ---
 * - Arduino (or compatible board)
 * - nRF24L01 Radio Module
 * - Servo Motor
 * - 8x8 Dot Matrix Display with MAX7219 driver (or shift registers)
 * - IR Receiver
 * - IR Remote
 */

//=============================================================================
// LIBRARIES
//=============================================================================
#include <SPI.h>          // Required for nRF24L01 communication
#include <RF24.h>         // The main library for the nRF24L01 radio module
#include <Servo.h>        // For controlling the servo motor
#include <IRremote.hpp>   // For receiving signals from the IR remote
#include <DotMatrix.h>    // A custom library for the 8x8 dot matrix display

//=============================================================================
// PIN & CONSTANT DEFINITIONS
//=============================================================================

// --- nRF24L01 Radio Pins ---
#define CE_PIN 7          // Chip Enable pin for the radio module
#define CSN_PIN 8         // Chip Select Not pin for the radio module

// --- Component Pins ---
#define SERVO_DOOR_PIN 6  // PWM pin connected to the servo motor signal wire
#define IR_RECEIVE_PIN 2  // Pin connected to the IR receiver's data out

// --- Dot Matrix Shift Register Pins ---
const int dataPin  = 5;   // Serial Data In (DS) of the 74HC595
const int clockPin = 3;   // Shift Register Clock (SHCP) of the 74HC595
```

```
48  const int latchPin = 4;   // Storage Register Clock (STCP) of the 74HC595
49
50  // --- IR Remote Button HEX Codes ---
51  // These values are specific to the IR remote being used.
52  #define BTN_UP     0x18
53  #define BTN_DOWN   0x52
54  #define BTN_LEFT   0x08
55  #define BTN_RIGHT  0x5A
56  #define BTN_CENTER 0x1C
57
58  //============================================================================
59  // GLOBAL VARIABLES & DATA STRUCTURES
60  //============================================================================
61
62  // --- RF & Hardware Objects ---
63  RF24 radio(CE_PIN, CSN_PIN);                  // Create an RF24 object
64  const byte pipeAddresses[][6] = {"00001", "00002"}; // Unique addresses for writing and
    reading pipes
65  Servo servoDoor;                              // Create a Servo object
66  DotMatrix matrix(dataPin, clockPin, latchPin); // Create a DotMatrix display object
67
68  // --- State Management ---
69  bool isVaultOpen = false; // Tracks the current state of the vault (locked/unlocked)
70
71  /**
72   * @enum RequestReason
73   * @brief Defines the purpose of a request sent to the server.
74   * This ensures the server knows how to interpret the incoming data.
75   */
76  enum RequestReason : uint8_t {
77    AUTH_REQUEST = 0,             // Password validation
78    DISARM_REQUEST = 1,           // Request to disarm server-side sensors
79    ARM_REQUEST = 2,              // Request to re-arm server-side sensors
80    RESET_REQUEST = 3,            // Request to reset the server's state
81    DOT_MATRIX_AUTH_REQUEST = 4,  // 2FA game coordinates validation
82    OPEN_LOCK_REQUEST = 5,        // Request for the server to open its lock
83    CLOSE_LOCK_REQUEST = 6,       // Command for the server to close its lock
84    INITIALIZATION = 7            // Initial message to let server know client is online
85  };
86
87  /**
88   * @struct RequestData
89   * @brief The data packet structure for communication between client and server.
90   * @note This structure MUST be identical on both the client and server code.
91   */
92  struct RequestData {
93    char password[8];       // Stores password or coordinates string
94    RequestReason reason;   // The reason for the request, from the enum above
95  };
96
97  //============================================================================
98  // SETUP FUNCTIONS
99  //============================================================================
100
```

```
/**
 * @brief Initializes the nRF24L01 radio module and sends an online notification.
 */
void radioSetup() {
  radio.begin();                          // Initialize the radio module
  radio.setChannel(108);                  // Set the channel (must match server)
  radio.openWritingPipe(pipeAddresses[0]); // Set the address to send data to the server
  radio.openReadingPipe(1, pipeAddresses[1]); // Set the address to receive data from the server
  radio.setPALevel(RF24_PA_MIN);          // Use minimum power for short-range communication
  radio.startListening();                 // Start listening for incoming data

  // Notify the server that this client is online
  RequestData initreq;
  initreq.reason = INITIALIZATION;
  sendServerRequest(initreq);
}

/**
 * @brief Main setup function, runs once at startup.
 */
void setup() {
  Serial.begin(9600);           // Start serial communication for debugging
  dotMatrixWithIrSetup();       // Configure pins for the dot matrix and IR receiver
  SPI.begin();                  // Initialize the SPI bus

  // Configure and initialize the servo motor
  servoDoor.attach(SERVO_DOOR_PIN);
  servoDoor.write(0);           // Start with the vault door closed

  // Initialize and configure the radio
  radioSetup();

  Serial.println("Client online. Enter password, 'close', or 'reset'.");

  // Configure the dot matrix display
  matrix.setSpeed(1);          // Set the scrolling speed
  matrix.setText("PSWRD");     // Display initial prompt
}

/**
 * @brief Configures pins and starts the IR receiver.
 */
void dotMatrixWithIrSetup() {
  IrReceiver.begin(IR_RECEIVE_PIN, ENABLE_LED_FEEDBACK); // Start the IR receiver
  pinMode(dataPin, OUTPUT);
  pinMode(clockPin, OUTPUT);
  pinMode(latchPin, OUTPUT);
}


//=======================================================================
// MAIN LOOP
```

```
//================================================================================

/**
 * @brief The main loop that runs continuously.
 */
void loop() {
  matrix.update(); // Continuously refresh the dot matrix display

  // --- Task 1: Listen for emergency commands from the server ---
  if (radio.available()) {
    char command[32] = "";
    radio.read(&command, sizeof(command));
    Serial.print("RF_RX: Received command: ");
    Serial.println(command);

    // Check if the server issued an emergency lockdown command
    if (strcmp(command, "LOCKDOWN") == 0) {
      Serial.println("LOCKDOWN received! Forcing vault lock.");
      closeVault();
    }
  }

  // --- Task 2: Listen for user input from the Serial Monitor ---
  if (Serial.available() > 0) {
    String userInput = Serial.readString();
    userInput.trim(); // Remove any whitespace

    if (userInput == "close") {
      matrix.setText("CL");
      closeVault();
      matrix.setText("PSWRD");
    } else if (userInput == "reset") {
      matrix.setText("RS");
      Serial.println("Sending RESET command to server...");
      RequestData req;
      req.reason = RESET_REQUEST;
      sendServerRequest(req);
      matrix.setText("PSWRD");
    } else if (!isVaultOpen) {
      // If the vault is closed, treat input as a password attempt
      sendPasswordForValidation(userInput);
    }
  }
}

//================================================================================
// AUTHENTICATION & COMMUNICATION
//================================================================================

/**
 * @brief Sends a password to the server and handles the two-factor auth process.
 * @param password The password entered by the user.
 */
void sendPasswordForValidation(String password) {
```

```
  // 1. Prepare the password request packet
  RequestData req;
  req.reason = AUTH_REQUEST;
  strncpy(req.password, password.c_str(), 8); // Copy password into the packet

  // 2. Send request and get the server's response
  char* response = sendServerRequest(req);

  // 3. Process the response
  if (strcmp(response, "VALID") == 0) {
    Serial.println("Password OK. Starting Dot Matrix Game...");
    matrix.setText("OK");
    unsigned long startTime = millis();
    while (millis() - startTime < 2000) { // Display "OK" for 2 seconds
      matrix.update();
    }

    // 4. Start the second factor of authentication (the game)
    if (startDotMatrixGameAuthentication()) {
      matrix.setText("OPEN");
      Serial.println("Access Granted!");

      // Tell the server to disarm its security sensors
      RequestData disarmReq;
      disarmReq.reason = DISARM_REQUEST;
      sendServerRequest(disarmReq);

      // Finally, open the vault
      openVault();
    } else {
      matrix.setText("PSWRD");
      Serial.println("Access Denied. Dot Matrix Authentication Failed.");
    }
  } else {
    Serial.println("Access Denied. Password authentication failed.");
  }
}

/**
 * @brief Manages the 2FA game where the user moves a dot with an IR remote.
 * @return True if the user selects the correct coordinates, false otherwise.
 */
bool startDotMatrixGameAuthentication() {
  int posX = 0, posY = 0; // Initial position of the dot (top-left)
  showDot(posX, posY);

  // Clear any pending IR signals
  while (IrReceiver.decode()) {
    IrReceiver.resume();
  }

  while (true) {
    // Wait for an IR signal
    if (IrReceiver.decode()) {
```

```
      uint8_t cmd = IrReceiver.decodedIRData.command;

      // Process the command only if it's not a repeat signal (from holding the button)
      if (!(IrReceiver.decodedIRData.flags & IRDATA_FLAGS_IS_REPEAT)) {
        switch (cmd) {
          // Move the dot based on the button pressed, constraining it to the 8x8 grid
          case BTN_DOWN:  posY = constrain(posY - 1, 0, 7); break;
          case BTN_UP:    posY = constrain(posY + 1, 0, 7); break;
          case BTN_LEFT:  posX = constrain(posX - 1, 0, 7); break;
          case BTN_RIGHT: posX = constrain(posX + 1, 0, 7); break;
          case BTN_CENTER:
            // User has selected their final coordinates
            RequestData req;
            req.reason = DOT_MATRIX_AUTH_REQUEST;
            sprintf(req.password, "%d|%d", posX, posY); // Format coordinates as "X|Y"

            // Send coordinates to server for validation
            char* response = sendServerRequest(req);
            showDot(0, 0); // Clear the dot

            // Return true if the server responds with "VALID", false otherwise
            return (strcmp(response, "VALID") == 0);
        }
      }
      IrReceiver.resume(); // Prepare for the next IR signal
    }
    showDot(posX, posY); // Refresh the dot's position on the display
  }
  return false; // This line is technically unreachable but good practice
}

/**
 * @brief A generic function to send any request to the server and wait for a reply.
 * @param req The RequestData struct to send.
 * @return A C-string with the server's response. Returns "TIMEOUT" on failure.
 * @note Returns a pointer to a static char array; the content is overwritten on each
call.
 */
char* sendServerRequest(RequestData req) {
  static char response[32] = "";
  memset(response, 0, sizeof(response)); // Clear the previous response

  // Temporarily switch radio to transmitting mode
  radio.stopListening();
  radio.write(&req, sizeof(RequestData));

  // Switch back to listening for the reply
  radio.startListening();

  // Wait for a response with a timeout
  unsigned long started_waiting_at = millis();
  while (!radio.available()) {
    if (millis() - started_waiting_at > 2000) { // 2-second timeout
      strcpy(response, "TIMEOUT");
```

```
      return response;
    }
  }

  // Read the response from the server
  radio.read(&response, sizeof(response));
  return response;
}

//================================================================================
// VAULT & DISPLAY CONTROL
//================================================================================

/**
 * @brief Opens the vault door by commanding the server lock and local servo.
 */
void openVault() {
  if (isVaultOpen) {
    Serial.println("Vault is already open.");
    return;
  }
  // 1. Tell the Server to unlock its side of the mechanism
  Serial.println("Sending OPEN_LOCK command to server...");
  RequestData req;
  req.reason = OPEN_LOCK_REQUEST;
  sendServerRequest(req);

  // 2. Open the client-side servo door
  Serial.println("Opening client-side door...");
  for (int pos = 0; pos <= 180; pos++) {
    servoDoor.write(pos);
    delay(15);
  }
  isVaultOpen = true;
  Serial.println("Vault is open.");
}

/**
 * @brief Closes the vault door and tells the server to re-arm security.
 */
void closeVault() {
  if (!isVaultOpen) {
    Serial.println("Vault is already locked and secured.");
    return;
  }
  // 1. Close the client-side servo door
  Serial.println("Closing client-side door...");
  for (int pos = 180; pos >= 0; pos--) {
    servoDoor.write(pos);
    matrix.update(); // Keep the display active during the process
    delay(15);
  }

  // 2. Tell the server to lock its side of the mechanism
```

```
368   Serial.println("Sending CLOSE_LOCK command to server...");
369   RequestData lockReq;
370   lockReq.reason = CLOSE_LOCK_REQUEST;
371   sendServerRequest(lockReq);
372
373   isVaultOpen = false;
374   Serial.println("Vault is closed.");
375
376   // 3. Tell the server to re-arm its security sensors
377   RequestData armReq;
378   armReq.reason = ARM_REQUEST;
379   sendServerRequest(armReq);
380 }
381
382
383 /**
384  * @brief Lights up a single pixel (dot) on the 8x8 matrix.
385  * @param x The column of the dot (0-7).
386  * @param y The row of the dot (0-7).
387  */
388 void showDot(byte x, byte y) {
389   byte rowData = (1 << y);  // Set the bit corresponding to the active row
390   byte colData = ~(1 << x); // Clear the bit for the active column (common cathode)
391   shiftOutData(rowData, colData);
392 }
393
394 /**
395  * @brief Sends row and column data to the shift registers to control the matrix.
396  * @param rowData The byte representing which row to activate.
397  * @param colData The byte representing which columns to activate.
398  */
399 void shiftOutData(byte rowData, byte colData) {
400   digitalWrite(latchPin, LOW); // Pull latch low to start data transfer
401   // Shift out column data then row data
402   shiftOut(dataPin, clockPin, MSBFIRST, colData);
403   shiftOut(dataPin, clockPin, MSBFIRST, rowData);
404   digitalWrite(latchPin, HIGH); // Pull latch high to apply the data
405 }
```